

SOBRE O PROJETO

App Android para gerenciar Ordens de Serviço de técnicos em campo. Funciona 100% offline com sincronização automática ao reconectar a internet. Permite criar OS, registrar defeitos e laudos, tirar fotos, coletar assinaturas digitais, capturar GPS e exportar relatórios em PDF e Word.

45+ Telas e widgets

8.500+ Linhas de código

9 Tabelas SQLite

10 Tabelas Supabase

100% Offline-First

LINGUAGENS DE PROGRAMAÇÃO

Dart

App mobile Android (100% do app)

Python

Sistema web Flask + geração PDF/Word

SQL (PostgreSQL)

Banco Supabase tabelas, RLS, triggers

XML

AndroidManifest + recursos nativos

STACK TÉCNICO — O QUE CADA PARTE FAZ

Bibliotecas agrupadas por FUNÇÃO para facilitar o entendimento.

1. Banco de Dados e Armazenamento

Onde os dados ficam guardados

supabase_flutter

Banco de dados na NUVEM (PostgreSQL). Online, todos os dados vão para cá e ficam disponíveis em qualquer dispositivo.

sqflite

Banco LOCAL no celular/tablet (SQLite). Guarda os dados offline para uso sem internet. 9 tabelas, versão 7.

path_provider

Encontra a pasta correta no dispositivo para salvar arquivos como fotos e PDFs gerados.

path

Monta caminhos de arquivo corretamente no Android (ex: /storage/emulated/0/...).

2. Conectividade e Sincronização

Detectar internet e enviar dados pendentes

connectivity_plus

Verifica em tempo real se o dispositivo tem internet. É chamado antes de qualquer operação de rede.

SyncService (interno)

Fila de sincronização criada no projeto. Guarda alterações feitas offline e envia ao Supabase automaticamente ao reconectar.

3. Autenticação e Segurança

Login online e offline com segurança

supabase_flutter (Auth)

Login com e-mail e senha via Supabase. Gera token JWT para autenticação segura online.

flutter_secure_storage

Guarda credenciais no dispositivo com criptografia forte. Permite login offline sem internet.

crypto	Hash SHA-256 da senha para verificacao offline — a senha nunca fica em texto puro no dispositivo.
---------------	---

4. Geracao de Documentos		Criar PDF e Word dentro do app, sem internet
pdf	Gera arquivos PDF profissionais direto no app Android. Layout com cores, tabelas, logos e imagens de assinatura.	
printing	Abre o menu de compartilhamento para enviar o PDF por WhatsApp, e-mail, Drive, Bluetooth, etc.	
archive	Gera arquivos Word (.docx). O formato Word e internamente um ZIP com XMLs — esta biblioteca monta esse ZIP.	
share_plus	Compartilha qualquer arquivo (PDF, Word, imagem) com outros aplicativos instalados no dispositivo.	

5. Camera, GPS e Permissoes		Funcionalidades de hardware do dispositivo
image_picker	Abre a camera ou galeria do dispositivo para o tecnico adicionar fotos nas OS.	
geolocator	Captura latitude e longitude da localizacao atual. Usa o GPS do dispositivo sem precisar de internet.	
permission_handler	Solicita as permissoes necessarias ao usuario em tempo de execucao: camera, localizacao, armazenamento.	
cached_network_image	Baixa fotos das OS e guarda em cache local. Apos a primeira visualizacao, funcionam offline.	

6. Interface e Experiencia do Usuario		O que o usuario ve e como o app se comporta
flutter + Material 3	Framework principal. Tudo que o usuario ve — telas, botoes, cards, animacoes — e construido com Flutter.	
flutter_localizations	Garante que datas, horas e textos aparecem em portugues (ex: "janeiro" em vez de "january").	
intl	Formata datas no padrao brasileiro (dd/MM/yyyy), horas e numeros com separador correto.	
uuid	Gera IDs unicos para OS, clientes e equipamentos criados offline, evitando conflitos ao sincronizar.	
flutter_native_splash	Splash screen nativa exibida imediatamente ao abrir o app, antes do Flutter carregar — sem tela branca.	
flutter_launcher_icons	Gera o icone do app em todos os tamanhos e densidades necessarios para Android.	

ARQUITETURA — COMO O APP FOI CONSTRUIDO

PADRAO PRINCIPAL: OFFLINE-FIRST

Toda tela e servico seguem a mesma logica antes de qualquer operacao:

1 Verifica conectividade	SEM INTERNET ► SQLite local (instantaneo)	COM INTERNET ► Supabase (nuvem)	ERRO DE REDE ► SQLite local (fallback)
--------------------------	---	---------------------------------	--

ESTRUTURA DE CAMADAS

Screens (Telas)	O que o usuario ve e toca. Cada funcionalidade tem sua propria tela separada.
Services (Servicos)	A logica do app. Decide se busca dados do SQLite ou do Supabase conforme a conexao.
OfflineService	Unico ponto de acesso ao SQLite. Todas as leituras e escritas locais passam por aqui.
SyncService	Roda em background. Monitora a conexao e envia dados pendentes ao Supabase automaticamente.
Models (Modelos)	Representacao dos dados: OS, Cliente, Equipamento, Usuario, Checklist, etc.
Supabase (Nuvem)	Banco central com RLS por usuario, autenticao JWT e armazenamento de fotos.
SQLite (Local)	9 tabelas offline espelhando os dados da nuvem. Versao 7 com migracao automatica entre versoes.

DESENVOLVIDO POR BRUNO (@YEN)

Todo o projeto foi idealizado, construido e executado pelo desenvolvedor:

Design do App	Identidade visual completa: tema dark green, paleta de cores, tipografia e layout de todas as telas e componentes.
UI / UX	Experiencia do usuario: fluxo de navegacao, cards, badges, animacoes da splash screen e assinatura em modo paisagem.
Codigo Flutter	Escrita e ajuste do codigo Dart, construcao das telas, formularios, logica e comportamentos do app.
Funcionalidades	Definicao de todos os requisitos: OS, checklist, GPS, fotos, assinaturas, PDF, Word e modo offline.
Integracao Supabase	Configuracao do projeto, tabelas, politicas de seguranca (RLS) e autenticao de usuarios.
Testes via terminal	Execucao de comandos no terminal: flutter run, flutter build apk, dart run e analise de erros e warnings.
Geracao de APK	Build e configuracao do APK de release para instalacao em dispositivos Android reais.

Testes em dispositivos	Validacao do app em tablet e celular — bugs visuais, travamentos e comportamentos inesperados.
Estabilidade do app	Identificacao e correcao de travamentos, spinners infinitos e problemas de performance em uso real.
Decisoões do produto	O que entra ou nao no app, prioridades, o que funciona offline e a experiencia do tecnico em campo.

INTELIENCIA ARTIFICIAL NO DESENVOLVIMENTO

Claude Code (Anthropic)

Modelo: Claude Sonnet 4.6 | Interface: CLI

O desenvolvimento contou com apoio da IA Claude Code nas seguintes areas:

Codigo Flutter/Dart	Implementacao completa de todos os 45+ arquivos Dart do projeto.
SQL / Supabase	Schema completo, RLS policies, functions, triggers e storage.
Logica Offline	SQLite schema, sync queue e offline-first em todos os servicos.
PDF e Word	Geradores completos em Dart (pdf + archive) e Python (reportlab + docx).
Bugs e auditoria	Analise e correcao de todos os bugs encontrados ao longo do projeto.
Performance	Otimizacao offline-first: de timeout 30 segundos para 0ms sem internet.

FERRAMENTAS UTILIZADAS

Android Studio	IDE principal para escrever, testar e depurar o codigo Flutter/Android.
Claude Code CLI	Interface da IA Claude no terminal do Android Studio para co-desenvolvimento.
Supabase Dashboard	Painel web para gerenciar banco, usuarios, storage e politicas de seguranca.
Git	Controle de versao — historico de todas as alteracoes do projeto.
Python 3	Linguagem do sistema web e dos scripts de geracao de PDF/Word e deste portfolio.
ViaCEP (API)	API gratuita que retorna endereco completo a partir do CEP digitado pelo usuario.
Windows 10	Sistema operacional do ambiente de desenvolvimento.